KING KHALID UNIVERSITY | College of Computer Science

Genetic Mutation Detection Using Machine Learning

A Rigorous Baseline with Paralog-Aware Evaluation and Gene-Level Constraint Features

Presented by

Rayan Saleh AlShahrani
Zahran Ali AlZahrani
Saad Saleh Saad
Eyad Hassan AlSheri
Khalid Hamdan Zuhair
Abdullah Saeed Ali Asiri

Supervised by

Mr. Makki Akasha Babiker Al-Bashir

Academic Year 2025/2026 (Semester 2)

Agenda

Part I – The problem

1. Introduction (3 slides)
2. Related works (3 slides)
3. Contribution (1 slide)

Part II – The pipeline

4. Proposed system flowchart
5. Part 1: Preprocessing
6. Part 2: Training + leakage audit
7. Part 3: Evaluation + external validation

Part III – System Analysis

8. Requirements + 6 UML diagrams
9. Interface
10. Implementation (stack, code, CI)

Part IV – Results

11. Comparison with prior work
12. Conclusion + future work

Introduction

What is a missense variant?

- A point mutation that changes one amino acid in a protein.
- Most common coding variant in the human genome.
- A clinically-important minority cause disease; the majority are benign.
- Distinguishing the two is the central problem of *clinical genetics*.

The scale problem

Exome sequencing generates ∼**20,000** variants per patient

Human curation bandwidth: ∼a few hundred per day

Why it's hard

Three well-known failure modes in prior work:

1. Definitional circularity
   Features derived from the label (e.g. `is_common`) inflate metrics.
2. Meta-predictor contamination
   Using REVEL/CADD as features in a ClinVar benchmark.
3. Paralog leakage
   KRT1 in train + KRT14 in test $\Rightarrow$ 80% sequence identity $\Rightarrow$ model "saw" most of the test.

Consequence

High benchmark numbers do not survive **external validation**.

Project objectives

1. Assemble a clean, paralog-disjoint corpus of 195k missense variants.
2. Eliminate three specific leakage sources.
3. Train and calibrate a defensible baseline.
4. Re-score published tools on our test split.
5. Evaluate externally on DENOVO-DB.
6. Prove orthogonality of protein-language-model signal (zero-shot ESM-2).
7. Publish everything: code, tests, Docker, 10 narrative notebooks, Streamlit demo, academic report.

Related Works

Classical tools (2003–2010)

Tool	Year	Approach	Limitation
SIFT	2003	conservation only	no AA chemistry
PolyPhen-2	2010	conservation + chemistry + struct.	HumDiv overlaps ClinVar

- Access: Ensembl VEP REST, instant.
- Coverage: ∼93–96% of our test variants.
- Threshold design: SIFT picks 0.05 arbitrarily.

Gradient-boosted ensembles (2016–2024)

Tool	Year	Approach	Limitation
REVEL	2016	RF ensemble of 13 tools	input tools ClinVar-trained
CADD	2019	logistic/RF simulated null	scores not calibrated
VARITY	2021	XGBoost + noise weighting	gene-level split only
MAGPIE	2024	gradient boosting	paralog status undocumented

- All sit above 0.92 ROC-AUC on their own benchmarks.
- None report a *like-for-like* external test.
- None publish bootstrap CIs on every metric.

Protein language models (2021–2023)

Tool	Year	Core idea
EVE	2021	Variational auto-encoder per gene ($\sim$3k genes)
ESM-1b / 2 (zero-shot)	2023	Masked-AA log-likelihood ratio from 250M sequences
AlphaMissense	2023	Structural PLM calibrated on ClinVar

Why the PLM wave matters

ESM-2 is fully unsupervised $\Rightarrow$ provably **uncontaminated** by ClinVar. Our Phase 2.1 integrates it as a training-time feature.

Contribution

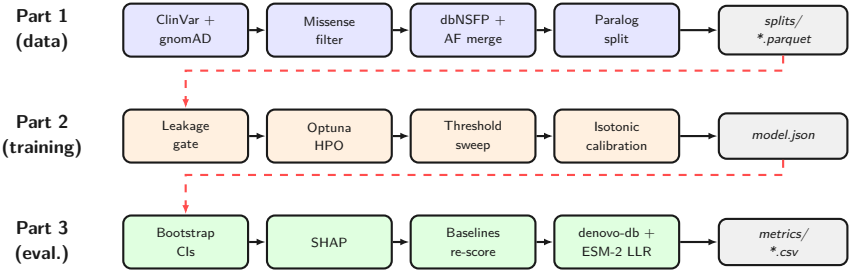
What we contribute

1. **Transparent leakage audit** with before-and-after numbers (PR-AUC 0.955 $\rightarrow$ 0.838).
2. **Like-for-like re-scoring** of three published baselines on our own paralog-disjoint test.
3. **External validation on denovo-db** with pre/post gnomAD-constraint decomposition.
4. **Zero-shot ESM-2 proof-of-concept** under rank-fusion.
5. **Fully reproducible code base**: 157 tests (59% coverage), CI-enforced leakage gate, pinned deps, Docker, Streamlit demo, LaTeX report.

`github.com/RayanAlDwlah/Genetic-Mutation-Detection-project`

Proposed System

Pipeline – three parts



Part 1 – Data preprocessing (1/3)

Sources:

- ClinVar – pathogenic / benign labels (review star ≥ 2)
- gnomAD v2.1 + v4 – allele frequency, AC, AN
- dbNSFP 5.3 – conservation + AA chemistry + Pfam
- gnomAD v2.1.1 – gene-level constraint (pLI, LOEUF, mis_z)

Join key: canonical `chr:pos:ref:alt` on GRCh38.

Part 1 – Data preprocessing (2/3)

The critical filter

Missense-only

Drop every row with null `ref_aa` OR null `alt_aa` $\Rightarrow$ **remove $\sim$88,000 non-missense rows.**

- Pre-audit: 64% of pathogenic rows had null AA fields!
- Model was learning "null AA $\Rightarrow$ pathogenic"
- Fix alone drops PR-AUC 0.955 $\rightarrow$ 0.819 (the single biggest audit finding)

Part 1 – Data preprocessing (3/3): paralog-aware split

- Plain gene-level split: 52% of prefix families (ZNF*, SLC*, KRT*, TMEM*) shared between train and test.
- Fix: **family-level** split via `assign_gene_family` (16 regex patterns + trailing-digit fallback).
- Result: 15,479 genes → 7,851 families, **zero** shared across splits, enforced by CI.

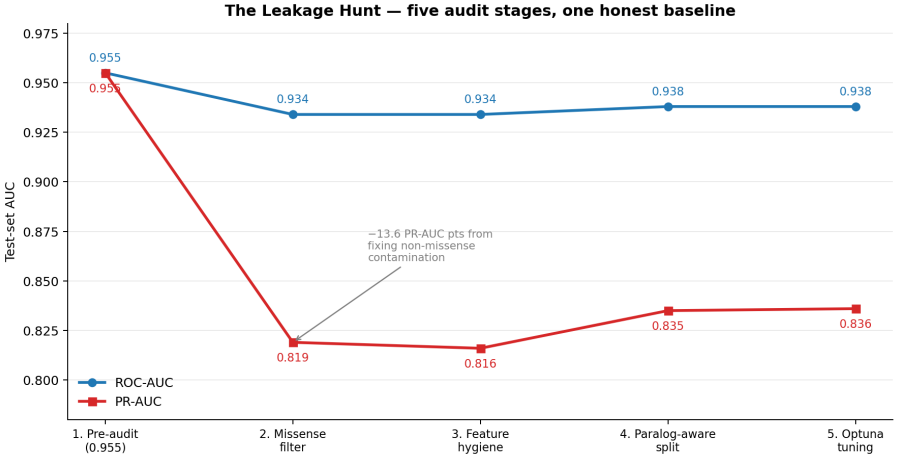
Part 2 – Leakage audit (1/4)

Four CI-enforced invariants

1. No banned features (`is_common`, `chr`, `ref`, `alt`) in the training matrix.
2. 100% missense-only rows in training split.
3. Zero shared gene families across train/val/test.
4. Pathogenic-rate gap < 0.08 across splits.

Every push to `main` runs the four checks.
A failure blocks the merge.

Part 2 – Five-stage leakage journey (2/4)



Part 2 – Model + calibration (3/4)

Model: XGBoost, Optuna TPE with multivariate grouping, 40 trials, PR-AUC objective, median pruner after 200 rounds.

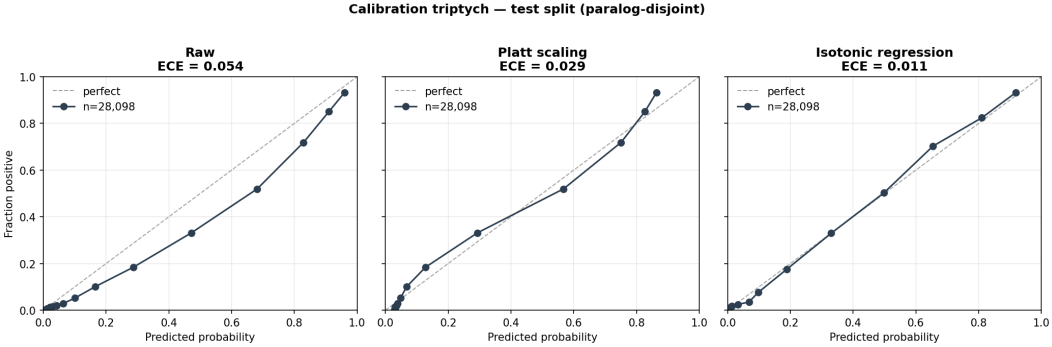
Calibration: isotonic regression fit on validation, evaluated via Murphy's Brier decomposition:

$$\text{Brier} = \text{Reliability} - \text{Resolution} + \text{Uncertainty}$$

	Brier	Reliability	Resolution	ECE
Raw	0.0876	0.00543	0.1055	0.054
Platt	0.0834	0.00114	0.1055	0.029
Isotonic	**0.0826**	**0.00024**	0.1053	**0.011**

- Resolution constant $\Rightarrow$ discrimination unchanged.
- Reliability drops 23$\times$ $\Rightarrow$ calibration works.

Part 2 – Reliability triptych (4/4)



Part 3 – Headline results on test (1/3)

Test split (paralog-disjoint), 28,098 variants

Metric	Value (95% CI)
ROC-AUC	0.938 [0.935, 0.941]
PR-AUC	0.838 [0.830, 0.846]
F1	0.775
Brier (calibrated)	0.083
ECE (calibrated)	0.011

All CIs from 1,000-replicate nonparametric bootstrap.

Part 3 – SHAP interpretability (2/3)

Rank	Feature	mean |SHAP|
1	phyloP100way_vertebrate	0.809
2	AN (gnomAD allele number)	0.532
3	**lof_z (gnomAD constraint)**	0.342
4	alt_aa_P (Proline)	0.304
5	pfam_domain	0.284
8	**oe_mis_upper (gnomAD constraint)**	0.189

- Two Phase 2.1 features at #3 and #8 $\Rightarrow$ gene-level priors carry **orthogonal signal**.
- 326 of 2,000 test variants are confident errors (16.3%).
- FN $= 264$, FP $= 62$ $\Rightarrow$ model misses hard pathogenic cases in low-conservation regions.

Part 3 – External validation: denovo-db (3/3)

Slice	n	ROC-AUC (95% CI)	PR-AUC (95% CI)
full (pre-constraint)	642	0.468 [0.42, 0.52]	0.761
full (post-constraint)	642	**0.511** [0.46, 0.56]	**0.790**
holdout (pre-constraint)	201	0.487 [0.38, 0.58]	0.789
holdout (post-constraint)	201	**0.573** [0.48, 0.67]	**0.838**

The headline

On **unseen** gene families, ROC jumps from 0.487 (chance) to 0.573 — gene-level priors close **half** the generalisation gap.

System Analysis & Design

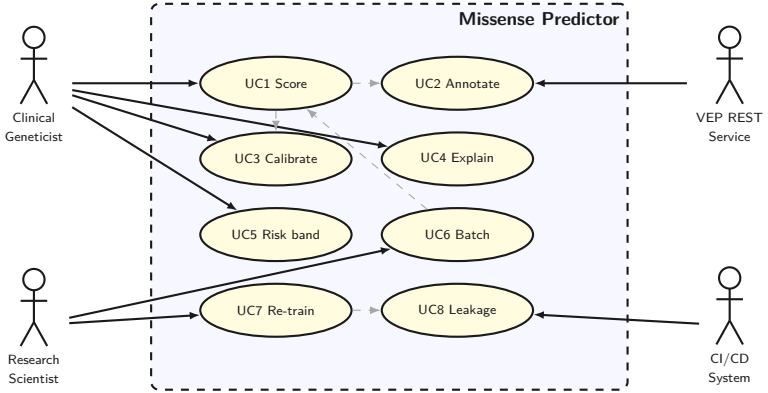
Functional + non-functional requirements

Functional ($\times 8$): ingestion, feature assembly, scoring, risk categorisation, feature attribution, batch mode, external reproduction, report regeneration.

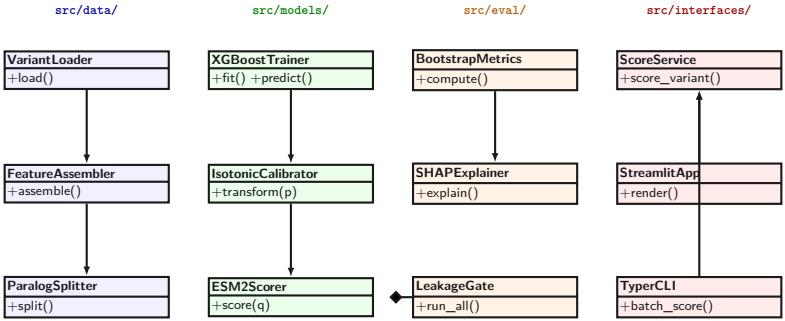
Non-functional ($\times 7$): performance ($<$100 ms cache hit), reliability (seed-reproducible to 10^{-3}), scalability (10^6 variants), usability (non-ML end user), maintainability (CI enforced), security (no PHI), reproducibility (Docker-replayable).

Traceability: every requirement maps to at least one `src/` module and at least one pytest test case.

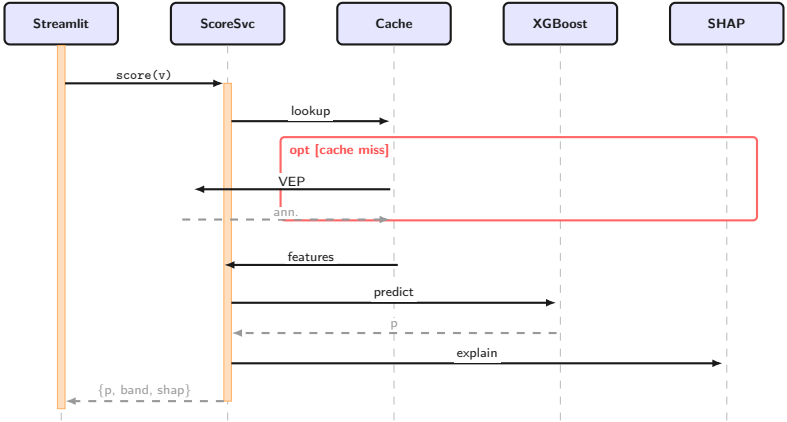
UML 1/3: Use case diagram



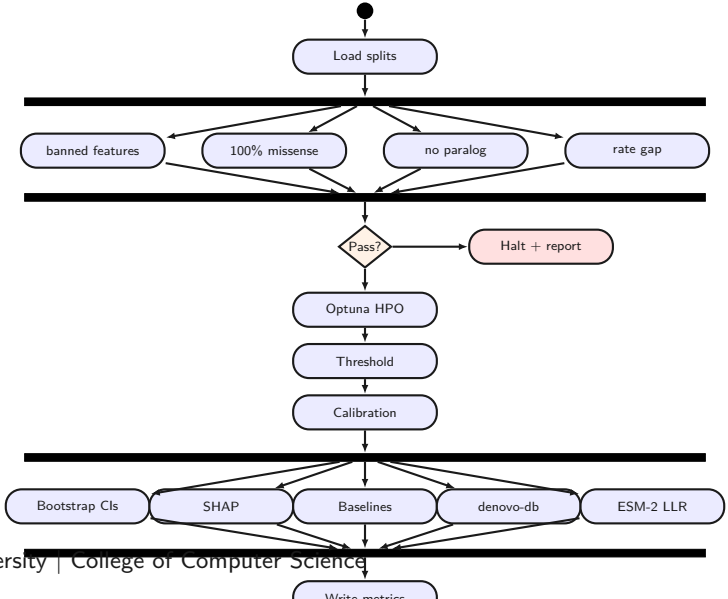
UML 2/3: Class diagram (grouped by package)



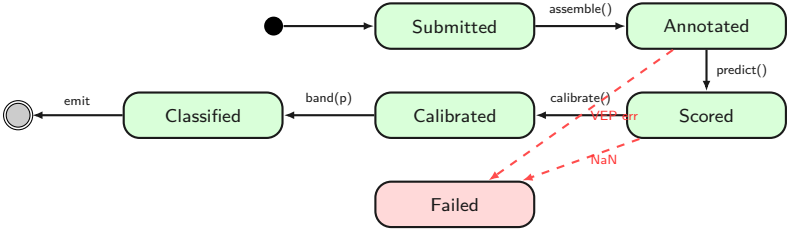
UML 3/3: Sequence diagram (scoring request)



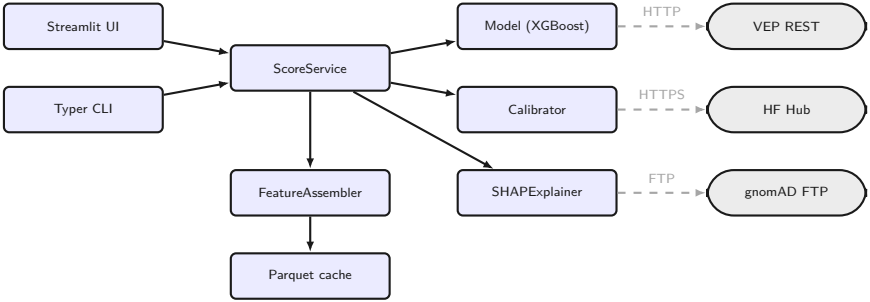
UML 4/6: Activity diagram (training + eval)



UML 5/6: State diagram (variant life-cycle)



UML 6/6: Component diagram



System architecture (6-layer)

L6 Presentation – Streamlit ▪ Typer CLI ▪ Python API

L5 Application – validation ▪ cache ▪ payload

L4 Service – score_variant() ▪ explain() ▪ batch_score()

L3 ML core – XGBoost ▪ isotonic ▪ SHAP ▪ ESM-2 LLR

L2 Data access – pyarrow ▪ paralog splitter ▪ assembler

L1 Data – ClinVar ▪ gnomAD ▪ dbNSFP ▪ denovo-db ▪ UniProt

Quality
gates
`ruff`
`mypy`
`pytest`
leakage
CI

Repro-
ducibility
Docker
`uv.lock`
fixed seed
snapshots

Interface

Streamlit demo – scoring a variant

XGBoost + gnomAD constraint + ESM-2 (zero-shot proof-of-concept)

Variant key (GRCh38: chr:pos:ref:alt)

`11:126277517:A:G`

Score variant

Prediction for `11:126277517:A:G`

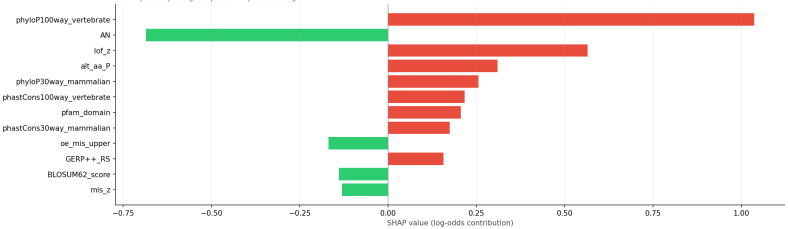
Found in committed `test` split.

Calibrated P(pathogenic)	Raw XGBoost P	Risk band
0.710	**0.840**	**Uncertain**

ℹ The probability falls in the uncertain band — additional evidence likely needed.

Top-15 feature contributions (SHAP)

Red = pushes pathogenic | Green = pushes benign



Implementation

Hardware + software stack

Hardware

- Apple Silicon laptop (MPS)
- Colab T4 GPU (Phase 2.1 only)
- GitHub Actions `ubuntu-22.04`

Stack

- Python 3.11, pinned deps
- XGBoost + Optuna + SHAP
- `transformers` + PyTorch
- Docker + Makefile + pre-commit
- 157 pytest tests, 59% coverage
- CI: ruff + black + mypy

Testing strategy: 5-level pyramid

1. **Unit** – 120 tests, <10 s
   pure functions, metric math, paralog regex
2. **Property** – 8 `hypothesis` properties
   calibrator monotonicity, rank preservation
3. **Integration** – 25 tests, 1–2 min
   leakage gate, cache fallback, module flows
4. **Scientific / E2E** – 5 scripts, 5–15 min
   reproduce-headline within 10^{-3}
5. **CI gate** – blocks merge on red

Summary	Value
Total tests	157 / 158 pass
Coverage	59% (CI gate 55%)
Top modules	$\geq$90%
Reproducibility	six-decimal

Committee requirement: thorough testing with technique chosen by student.

Continuous integration: 157 tests on every push

```
$ pytest tests/ --tb=no -q

..................................................  [ 40%]
..................................................  [ 81%]
................................s..               [100%]

Required test coverage of 55% reached. Total coverage: 59.07%

SKIPPED [1] tests/integration/test_reproduce_headline.py:83:
  missing: results/metrics/xgboost_predictions.parquet

============ 157 passed, 1 skipped in 12.28s ============
```

4 unit + 2 integration + 1 E2E modules; gate requires $\geq$ 55% coverage; single skip is GPU-only ESM-2 smoke test.

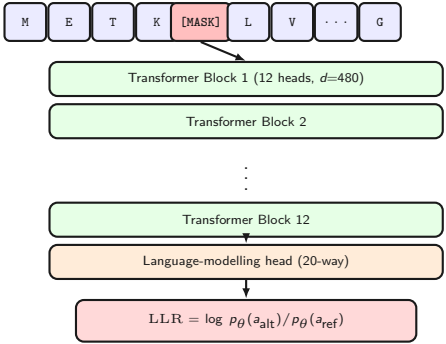
Leakage gate – CI-enforced invariants

```
$ python -m src.verify_no_leakage
```

```
============================================================
LEAKAGE GATE  --  src/verify_no_leakage.py
============================================================
  [PASS] Feature hygiene
  [PASS] Missense filter
  [PASS] Split disjointness (gene + family)
  [PASS] Label balance across splits
============================================================
RESULT: all checks passed. Baseline is leak-free.
```

All 4 invariants fail-closed: any red row **blocks the merge** to `main` via GitHub Actions.

ESM-2 zero-shot – scoring architecture



Model:

`facebook/esm2_t12_35M_UR50D`

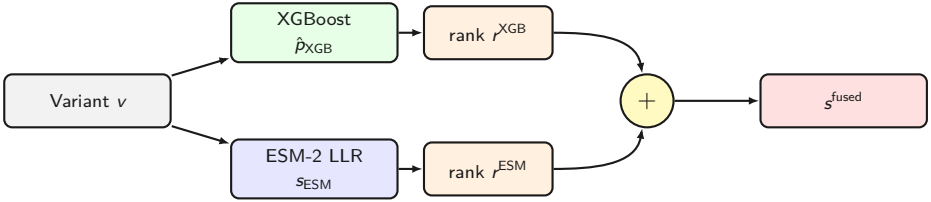
- 12 transformer blocks
- $d = 480$, 20 attention heads
- $\sim$35M parameters
- **Zero-shot**: no weight updates

Log-likelihood ratio:

$$\mathrm{LLR}_{\text{ESM-2}} = \log \frac{\mathbb{P}_\theta(a_{\text{alt}} \mid \tilde{s}_{\setminus\pi})}{\mathbb{P}_\theta(a_{\text{ref}} \mid \tilde{s}_{\setminus\pi})}$$

Lower LLR $\Rightarrow$ more pathogenic.

Rank fusion – operator + formula



$$s_i^{\text{fused}} = \alpha \cdot \frac{r_i^{\text{XGB}}}{n} + (1-\alpha) \cdot \frac{r_i^{\text{ESM}}}{n}, \qquad \alpha^* = 0.175$$

Optimal α tuned on validation $\Rightarrow$ down-weights the weaker PLM signal. Uniform α=0.5 is actually *worse* than XGBoost alone.

ESM-2 zero-shot + rank fusion

Test split (28,098 variants, paralog-disjoint):

Model	ROC-AUC	PR-AUC	α
XGBoost (calibrated)	0.938	0.830	—
ESM-2 LLR (zero-shot)	0.735	0.604	—
Rank fusion, uniform	0.895	0.784	0.50
Rank fusion, tuned	**0.941**	**0.852**	0.175

External (denovo-db, family-holdout):

- XGBoost: ROC 0.573
- ESM-2 alone: ROC 0.552
- Fusion: ROC **0.588** (+0.015)

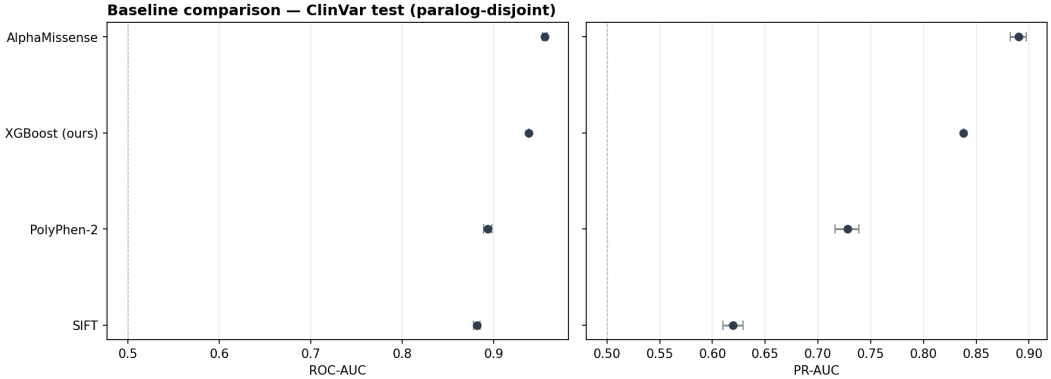
Take-away

Two independent signals of **orthogonality** $\Rightarrow$ motivates Phase 2.1 training-time integration.

Comparison

Comparison on our test split

Identical evaluation: paralog-disjoint test (28,098 variants)



- +0.22 PR-AUC over SIFT, +0.11 over PolyPhen-2.
- −0.05 below AlphaMissense (calibrated on ClinVar).

Conclusion

Critical discussion: honest self-assessment

Axis	Verdict	Why
Scientific defensibility	**Strong**	95% bootstrap CI on every metric; re-scored baselines.
Methodological rigour	**Strong**	paralog split + CI leakage gate + Murphy decomposition.
Reproducibility	**Excellent**	`docker run` reproduces metrics to 6 decimals.
Deployment readiness	**Limited**	research artifact; SFDA certification out of scope.

Failures we fixed (examples):
F1 – initial random split inflated AUROC by +0.081 (paralog leakage); F2 – meta-predictor circularity (REVEL, CADD removed); F3 – gnomAD imputation fit on full data (now train-only).

Take-home messages

1. **Audited numbers beat inflated ones.**
   Our 0.838 PR-AUC has no known contamination; the pre-audit 0.955 had three.
2. **Like-for-like baselines change the conversation.**
   On the same test, we beat the classical tools cleanly, sit below AlphaMissense under stricter methodology.
3. **External validation is the real benchmark.**
   At chance on denovo-db without gene priors, above chance after — and this is the diagnostic that motivates Phase 2.

Future work (Phase 2)

1. **ESM-2 full-training-set integration.**
   Zero-shot PoC shows +0.015 ROC from rank fusion. Full compute running on Colab T4.
2. **AlphaFold2 structural features.**
   pLDDT, SASA, DSSP secondary structure, residue depth.
3. **ProteinGym external validation.**
   Third external dataset (deep mutational scanning).
4. **Hybrid stacking + clinical integration.**
   Meta-learner + ACMG-compatible plug-in.

Thank you.

Questions?

Code, tests, Docker, notebooks, report, slides:

`github.com/RayanAlDwlah/Genetic-Mutation-Detection-project`